Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Laboratory Work No. 4:

Block Ciphers. The DES Algorithm

Task 2.4 — Computing L_1

Course: Cryptography and Security

Elaborated: st. gr. FAF-241 Lungu Ilie

Verified: asist. univ. *Zaica Maia*

Chişinău – 2026

Contents

1 SHORT THEORY **2**
- 1.1 Overall structure . . . 2
- 1.2 The function f . . . 2
- 1.3 Key schedule . . . 2

2 TASK **2**
- 2.1 Solution idea . . . 3

3 IMPLEMENTATION **3**
- 3.1 Verification . . . 3

4 RESULTS **4**
- 4.1 Test 1: first block of the handout message . . . 4
- 4.2 Test 2 . . . 4
- 4.3 Input validation . . . 4

5 CONCLUSIONS **4**

A Source code: `des_l1.py` **5**

1 SHORT THEORY

A block cipher encrypts a message in fixed-size blocks, applying the same keyed transformation to every block. The Data Encryption Standard (DES), published as FIPS 46, is a block cipher with a block size of 64 bits and a key of 64 bits, of which 56 bits are effective (every eighth bit is a parity bit). It is a *Feistel network*: each block is split into a left half L and a right half R of 32 bits, and every round computes

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus f(R_{i-1}, K_i),$$

where K_i is a 48-bit round key. Because of this structure, decryption uses the same algorithm with the round keys in reverse order.

1.1 Overall structure

Encryption of one 64-bit block M proceeds as follows.

1. **Initial permutation.** The bits of M are permuted with the table IP, and the result is split as $IP(M) = L_0R_0$ into two 32-bit halves.
2. **Sixteen rounds.** For $i = 1, \ldots, 16$ the Feistel step above is applied.
3. **Final permutation.** The halves are swapped, $R_{16}L_{16}$, and permuted with IP^{-1}; the result is the 64-bit ciphertext.

1.2 The function f

The function $f(R_{i-1}, K_i)$ has four stages:

1. the 32-bit R_{i-1} is expanded to 48 bits with the expansion table E;
2. the result is XORed with the round key: $K_i \oplus E(R_{i-1}) = B_1B_2 \ldots B_8$, eight blocks of 6 bits;
3. each B_j is replaced by a 4-bit value $S_j(B_j)$ taken from the substitution box S_j (the outer two bits of B_j select the row, the inner four bits select the column);
4. the 32-bit result $S_1(B_1) \ldots S_8(B_8)$ is permuted with the table P.

1.3 Key schedule

The 64-bit key is reduced to 56 bits with the permutation PC-1, giving K^+, which is split into halves C_0 and D_0 of 28 bits. For each round C_i and D_i are obtained by cyclic left shifts of C_{i-1} and D_{i-1} by 1 position (rounds 1, 2, 9, 16) or by 2 positions (all other rounds). The round key is $K_i = PC\text{-}2(C_iD_i)$, a 48-bit value.

2 TASK

According to the handout, the task is chosen by the formula task $= n \bmod 11$, where n is the student's number in the group list. For $n = 15$ this gives $15 \bmod 11 = 4$, i.e. **Task 2.4**: *given a message of 8 characters in the DES algorithm, find* L_1.

The program has to display the tables used and all intermediate steps, and the input must be either entered by the user or generated randomly.

2.1 Solution idea

Only the very beginning of the algorithm is needed. The 8 characters are 8 bytes = 64 bits, i.e. exactly one DES block M. After the initial permutation, $IP(M) = L_0R_0$. In the first round the Feistel rule gives

$$L_1 = R_0,$$

so L_1 is the right half (the last 32 bits) of $IP(M)$. Neither the key nor the function f is involved.

The initial permutation table IP is shown in Table 1; entry j says which input bit becomes output bit j (bits are numbered from 1 starting with the leftmost bit).

Table 1: Initial permutation IP.

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

3 IMPLEMENTATION

The program `des_l1.py` (Python 3, Appendix A) consists of the following functions.

- `validate` checks that the message has exactly 8 characters and that all of them are ASCII (code 0–127); otherwise it prints an error.
- `to_bits` converts every character to its 8-bit ASCII code and concatenates the codes into a 64-bit string.
- `permute` applies a permutation table to a bit string.
- `l1_of_block` performs the algorithm and prints every intermediate value: the ASCII codes, the hexadecimal form, the 64 message bits, the table IP, the permuted block, L_0, R_0 and finally $L_1 = R_0$ (in binary and hexadecimal).
- `main` offers a menu: enter a message manually, generate a random message of 8 letters and digits, or exit.

3.1 Verification

The permutation was checked on the standard test block from the DES literature, M = `0123456789ABCDEF` (hex). The program gives L_0 = `CC00CCFF` and R_0 = `F0AAF0AA`, the values given in the standard worked example, so the table IP and the bit numbering are correct.

4 RESULTS

4.1 Test 1: first block of the handout message

The message of the alternative task in the handout starts with "The Data Encryption Standard..."; its first block is `The Data` (the space is the character with code 32).

```
Message M      : The Data
ASCII codes    :  84 104 101  32  68  97 116  97
Hex            : 5468652044617461
M (64 bits)    : 01010100 01101000 01100101 00100000
                 01000100 01100001 01110100 01100001
IP(M)          : 11110111 01000001 01010101 10100100
                 00000000 11101110 00000010 00000000
L0 (32 bits)   : 11110111 01000001 01010101 10100100
R0 (32 bits)   : 00000000 11101110 00000010 00000000
L1 = R0        : 00000000 11101110 00000010 00000000  (hex 00EE0200)
```

Here $L_0 =$ `F74155A4` and $\mathbf{L_1} =$ `00EE0200`.

4.2 Test 2

Message: `Standard`.

```
Message M      : Standard
ASCII codes    :  83 116  97 110 100  97 114 100
Hex            : 5374616E64617264
M (64 bits)    : 01010011 01110100 01100001 01101110
                 01100100 01100001 01110010 01100100
IP(M)          : 11111111 01000011 10011010 00100101
                 00000000 11111110 00001000 01001001
L0 (32 bits)   : 11111111 01000011 10011010 00100101
R0 (32 bits)   : 00000000 11111110 00001000 01001001
L1 = R0        : 00000000 11111110 00001000 01001001  (hex 00FE0849)
```

Here $L_0 =$ `FF439A25` and $\mathbf{L_1} =$ `00FE0849`.

4.3 Input validation

A message of the wrong length is rejected with an explanatory message and the user is asked again, for example `short` produces "The message must have exactly 8 characters (now 5)." Non-ASCII characters are rejected as well.

5 CONCLUSIONS

Task 2.4 was solved by computing the initial permutation of an 8-character message and taking the right half of the result: since the first Feistel round sets $L_1 = R_0$, no key material or S-box computation is needed to obtain L_1. The program prints the table IP and all intermediate values, accepts both user input and random messages, and was checked against the standard DES test block.

The exercise also shows the structure of DES that the rest of the algorithm builds on: the initial permutation prepares the two halves, every round swaps them while mixing one half through f, and the key schedule supplies sixteen 48-bit round keys. Because $L_i = R_{i-1}$ at every round, the cipher is invertible even though f itself is not.

A Source code: `des_l1.py`

```
#!/usr/bin/env python3
"""Lab 4, Task 2.4 -- DES: given an 8-character message, find L1.

L1 = R0, where (L0, R0) = IP(M) is the split of the initial permutation.
"""

import random
import string

IP = [58, 50, 42, 34, 26, 18, 10, 2,
      60, 52, 44, 36, 28, 20, 12, 4,
      62, 54, 46, 38, 30, 22, 14, 6,
      64, 56, 48, 40, 32, 24, 16, 8,
      57, 49, 41, 33, 25, 17, 9, 1,
      59, 51, 43, 35, 27, 19, 11, 3,
      61, 53, 45, 37, 29, 21, 13, 5,
      63, 55, 47, 39, 31, 23, 15, 7]


def to_bits(text):
    """8 ASCII characters -> 64-bit string (8 bits per character)."""
    return "".join(format(ord(c), "08b") for c in text)


def group(bits, n=8):
    return " ".join(bits[i:i + n] for i in range(0, len(bits), n))


def permute(bits, table):
    return "".join(bits[i - 1] for i in table)


def to_hex(bits):
    return format(int(bits, 2), "0%dX" % (len(bits) // 4))


def validate(text):
    if len(text) != 8:
        raise ValueError("The message must have exactly 8 characters "
                         "(now %d)." % len(text))
    if any(ord(c) > 127 for c in text):
        raise ValueError("Only ASCII characters (code 0-127) are allowed.")
    return text


def print_table(title, table, cols=8):
    print(title)
    for i in range(0, len(table), cols):
        print("  " + " ".join("%2d" % x for x in table[i:i + cols]))


def l1_of_block(text, verbose=True):
    bits = to_bits(text)
    ip = permute(bits, IP)
    l0, r0 = ip[:32], ip[32:]
    l1 = r0
    if verbose:
        print("Message M      :", text)
        print("ASCII codes    :", " ".join("%3d" % ord(c) for c in text))
        print("Hex            :", to_hex(bits))
        print("M (64 bits)    :", group(bits))
        print_table("Initial permutation IP:", IP)
        print("IP(M)          :", group(ip))
        print("L0 (32 bits)   :", group(l0))
        print("R0 (32 bits)   :", group(r0))
```

```
        print("L1 = R0        :", group(l1), " (hex %s)" % to_hex(l1))
    return l1


def main():
    while True:
        choice = input("1 = enter message, 2 = random message, 0 = exit: ").strip()
        if choice == "0":
            break
        if choice == "2":
            text = "".join(random.choice(string.ascii_letters + string.digits)
                           for _ in range(8))
        elif choice == "1":
            text = input("Message (8 characters): ")
            try:
                validate(text)
            except ValueError as e:
                print("  Error:", e)
                continue
        else:
            print("  Error: choose 1, 2 or 0.")
            continue
        l1_of_block(text)


if __name__ == "__main__":
    main()
```